МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ИС

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Алгоритмы и структуры данных»
Тема: Алгоритмы сжатия без потерь

Студент(ка) гр. 4352 _____ Макарова Ю.И.

Преподаватель _____ Пестерев Д.О.

Санкт-Петербург
2026

Цель работы.

Знакомство с алгоритмами преобразования данных и сжатия

Теоретическая часть

Run-length encoding

Алгоритм заменяет последовательности одинаковых значений парой чисел: количество повторений и само значение.

Временная сложность: *O(n)* – требуется один проход по входящей строке

Пространственная сложность: *O(1)* – в лучшем случае, много одинаковых символов;

O(n) – в худшем случае, когда все символы разные

Энтропийное кодирование

<u>Функция расчета энтропии</u>

Алгоритм проходит по строке, создавая словарь уникальных символов, где ключ: символ и значение: количество повторений. Затем он проходит по словарю и по формуле $H = -\sum \frac{k}{N} \log_2 \frac{k}{N}$ накапливает энтропию строки.

Временная сложность: *O(n+m)* – требуется один проход по входящей строке + один проход по словарю

Пространственная сложность: *O(1)* – в лучшем случае, один уникальный символ;

O(n) – в худшем случае, все символы разные

<u>MTF</u>

Алгоритм поддерживает динамический список символов (обычно весь алфавит). Для каждого входного символа выполняются три шага:

1. Поиск: Найти индекс текущего символа в списке.
2. Вывод: Записать этот индекс в выходной поток.

3. Перемещение: Переместить найденный символ в начало списка (на позицию 0).

Временная сложность: *O(n)* – требуется один проход по входящей строке

Пространственная сложность: *O(1)* – храним только словарь символов

Алгоритм Хаффмана

Алгоритм присваивает каждому символу входного алфавита код переменной длины: часто встречающимся символам - короткие коды, редким - длинные.

1. Подсчет частот: Подсчитывается количество вхождений каждого символа во входных данных.
2. Построение дерева: Символы помещаются в очередь с приоритетом. Два элемента с наименьшей частотой объединяются в новый узел, который возвращается в очередь. Процесс повторяется, пока не останется один корневой узел.
3. Генерация кодов: Обход дерева от корня к листьям (влево - 0, вправо - 1). Путь до символа является его кодом.
4. Кодирование: Замена символов исходной строки на полученные битовые коды.

Для декодирования получателю необходимо передать структуру дерева Хаффмана вместе с данными. Это занимает дополнительную память и усложняет передачу.

Временная сложность: *O(n*log n)* – создание очереди – *O(n)*, построение дерева *O(n*log n)*, генерация кодов *O(n)* – обход дерева

Пространственная сложность: *O(n)* – очередь, дерево

Канонические коды Хаффмана

Модификация алгоритма, позволяющая восстановить таблицу кодов, зная только длины кодов для каждого символа, а не полную структуру дерева.

1. Строится обычное дерево Хаффмана и определяются длины кодов для каждого символа.
2. Символы сортируются сначала по длине кода (возрастание), затем по значению символа (лексикографически).
3. Коды назначаются последовательно: первый символ получает код из нулей длиной L, следующий код получается увеличением предыдущего на 1 и сдвигом влево при изменении длины.

Временная сложность: *O(n*log n)* – построение дерева *O(n*log n)*, сортировка символов *O(n*log n)*, присвоение кодов *O(n)*.

Пространственная сложность: *O(n)* – длины кодов

<u>Арифметическое кодирование</u>

Метод энтропийного сжатия, который кодирует всю последовательность данных в одно число (дробь) в интервале [0, 1).

1. Начальный интервал: [0, 1)
2. Для каждого символа: интервал делится пропорционально вероятностям символов: выбирается подынтервал, соответствующий текущему символу
3. Кодирование завершается - финальный интервал представляется в виде бинарной дроби

Временная сложность: *O(n)*

Пространственная сложность: *O(1)*

Преобразование Барроуза-Уиллера

BWT

<u>Прямое преобразование</u>

1. Сформировать все циклические сдвиги строки
2. Отсортировать сдвиги лексикографически

3. Выбрать последний столбец отсортированной матрицы, записать индекс исходной строки в матрице

Временная сложность: $O(n^2 \log n)$ – создание матрицы n^2 * сортировка строк + взятие последнего столбца n

Пространственная сложность: $O(n^2)$ – матрица

<u>Обратное преобразование (наивное)</u>

1. Начинаем с пустых строк
2. На каждом шаге i:

   Берем i-й символ из BWT-строки

   Добавляем его в начало каждой строки текущей матрицы

   Сортируем полученные строки лексикографически
3. После n итераций получаем полную матрицу
4. Возвращаем строку по индексу index (позиция исходной строки)

Временная сложность: $O(n^3 \log n)$ – на каждой итерации создание списка n * сортировка строк $n^2 \log n$

Пространственная сложность: $O(n^2)$ – матрица

<u>Обратное преобразование (с использованием порождаемой сдвигом и сортировкой перестановки)</u>

1. Строим массив next_pos — для каждой позиции в L указываем соответствующую позицию в F
2. Начинаем с index (позиция исходной строки в первом столбце)
3. Проходим по цепочке row = next_pos[row], собирая символы из последнего столбца
4. Разворачиваем полученную строку (так как идем с конца)

Временная сложность: O(n)

Пространственная сложность: O(n)

Алгоритм нахождения суффиксного массива (Prefix Doubling)

1. Начинаем с сортировки по первому символу (префикс длины 1)
2. Присваиваем ранги суффиксам на основе этой сортировки
3. На каждом шаге k сортируем суффиксы по парам:

   Первый ключ: текущий ранг суффикса (префикс длины k)

   Второй ключ: ранг суффикса, начинающегося через k позиций (префикс длины k)
4. После сортировки обновляем ранги
5. Удваиваем k и повторяем, пока все ранги не станут уникальными

Временная сложность: $O(n \log^2 n)$ – количество итераций log n * сортировка на каждой итерации n log n

Пространственная сложность: O(n)

Методы Лемпеля-Зива

Алгоритм LZ77 - словарный метод сжатия, использующий скользящее окно. Алгоритм заменяет повторяющиеся последовательности ссылками на предыдущие вхождения в формате (смещение, длина, следующий символ).

1. Скользящее окно делится на:

Буфер поиска (search buffer) — уже закодированные данные

Буфер просмотра (look-ahead buffer) — данные для кодирования

2. Для каждой позиции в буфере просмотра ищем максимальное совпадение в буфере поиска
3. Выдаем тройку: <смещение, длина, следующий символ>
4. Сдвигаем окно на *длина + 1* символов

Временная сложность: O(n * L * W), где n – количество символов, L – длина совпадения, W - подстрока в search buffer

Пространственная сложность: O(n)

Алгоритм LZSS - это улучшенная версия LZ77, которая:

1. Использует флаг для различения литералов и ссылок
2. Не кодирует следующий символ в ссылке (только смещение и длина)
3. Экономит место, не сохраняя лишний символ

Временная сложность: O(n * L * W), где n – количество символов, L – длина совпадения, W - подстрока в search buffer

Пространственная сложность: O(n)

Алгоритм LZ78 - словарный метод сжатия, который строит словарь фраз по мере кодирования. В отличие от LZ77 (скользящее окно), LZ78 использует глобальный словарь, который пополняется новыми фразами.

1. Словарь инициализируется пустой строкой (индекс 0)
2. Читаем данные, накапливая текущую фразу, пока она есть в словаре
3. Когда добавляем следующий символ и фразы нет в словаре:

   Выдаем пару <индекс префикса, следующий символ>

   Добавляем новую фразу в словарь
4. Сбрасываем текущую фразу и продолжаем

Временная сложность: O(n) – n итераций

Пространственная сложность: O(n+m), где m – размер словаря

Алгоритм LZW - это улучшенная версия LZ78, где:

1. Словарь инициализируется всеми возможными символами (0-255)
2. Кодируются только индексы (без дополнительного символа)
3. Словарь строится из уже встреченных фраз
4. Более эффективен, так как не нужно передавать следующий символ

Временная сложность: O(n) – n итераций

Пространственная сложность: O(n+m), где m – размер словаря

Экспериментальная часть

Задание 1

Исследовать зависимость энтропии сообщения от размера блоков, на которые разбивается текст, подаваемый на вход BWT+MTF, для тестовых файлов. Вывести полученные результаты на график, сравнить с исходной энтропией и сделать вывод об оптимальном размере блока.

Текст, подаваемый на вход, разбивался на блоки от 500 до 15000 байтов с шагом 1000.

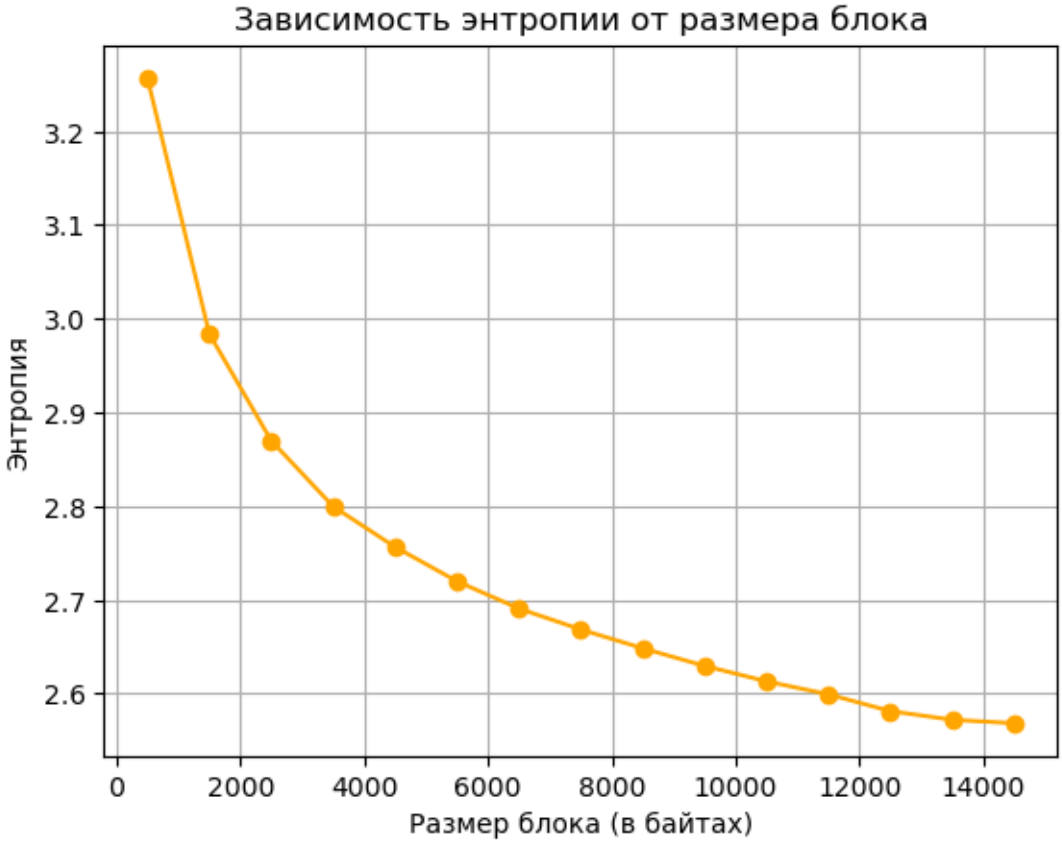

Рисунок 1 – Исследование зависимости энтропии от размера блоков

Проанализировав полученные результаты, мы можем сделать вывод, что с увеличением размера блоков уменьшается энтропия за счет появления длинных появления длинных последовательностей одинаковых символов. Диапазон оптимальных размеров блоков 8000-12000 байт:

<8000 – энтропия резко падает, оптимально сжимать дальше

>12000 – график меняется не так сильно, а ресурсные затраты возрастают

8000-12000 – хороший баланс сжатия и затрат

Задание 2

Исследовать зависимость коэффициента сжатия от размера буфера для алгоритма LZSS. Вывести полученные результаты на график и сделать вывод об оптимальном размере буфера.

Для исследования зависимости в функции было выставлено окно размером 4096 байта.

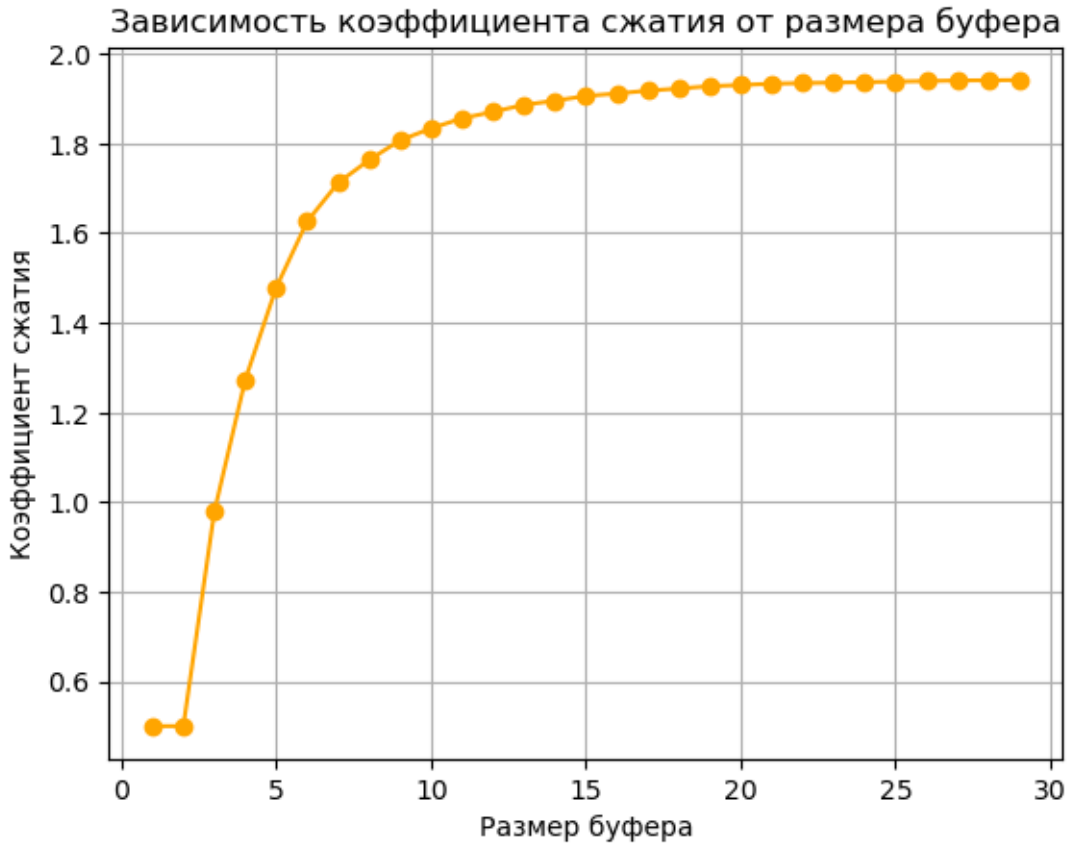

Рисунок 2 - Исследование зависимости коэффициента сжатия от размера буфера для LZSS

Проанализировав полученные результаты, мы можем сделать вывод, что оптимальный размер буфера – 10-15 байт:

<10 байт – коэффициент сжатия быстро растет

>15 байт – рост функции прекращается и начинается перерасход ресурсов

Максимальный коэффициент сжатия составляет 1.95. Выход на плато объясняется локальным характером повторений в тестовых данных.

Задание 3

Исследовать зависимость коэффициента сжатия от размера словаря для алгоритма LZW. Вывести полученные результаты на график и сделать вывод об оптимальном размере словаря

Рисунок 3 - Исследование зависимости коэффициента сжатия от размера словаря для алгоритма LZW

На графике мы видим плавный рост функции, означающий, что по мере увеличения словаря растет коэффициент сжатия, хотя и не сильно. Также коэффициент меньше 0,5 – это значит, что файл увеличился в 2 раза. Это могло произойти из-за фиксированного литерала в 2 байта - каждый код словаря кодируется двумя байтами (16 бит), тогда как исходные символы занимают по 1 байту. Без нахождения достаточно длинных повторяющихся последовательностей алгоритм лишь увеличивает размер данных.

Задание 4

Исследовать эффективность компрессоров для всех тестовых данных. Свести размеры исходных файлов, файлов после компрессии и файлов после декомпрессии в байтах и соответствующие коэффициенты сжатия в таблицу.

enwik7				
	Исходный размер	После компрессии	После декомпрессии	Коэффициент сжатия
HA	10000000	6412794	10000000	1.56
RLE		10374322		0.96
BWT + RLE		9039063		1.11
BWT + MTF + HA		4300804		2.33
BWT + MTF + RLE + HA		5319255		1.87
LZSS		9864125		1.01
LZSS + HA		6634886		1.5
LZW		19987636		0.5
LZW + HA		8916365		1.12
текст на русском языке, объемом не менее 200Кб				
	Исходный размер	После компрессии	После декомпрессии	Коэффициент сжатия
HA	371816	179927	371816	2.07
RLE		364470		0.99
BWT + RLE		217222		1.66
BWT + MTF + HA		124879		2.97
BWT + MTF + RLE + HA		117934		3.15
LZSS		252799		1.47

LZSS + HA		176782		2.1
LZW		722276		0.51
LZW + HA		274892		1.35
бинарный файл объемом не менее 1 Мб, содержимое которого не представляет собой осмысленный текст				
	Исходный размер	После компрессии	После декомпрессии	Коэффициент сжатия
HA	1104440	947680	1104440	1.17
RLE		1051847		1.05
BWT + RLE		912760		1.21
BWT + MTF + HA		640538		1.72
BWT + MTF + RLE + HA		683098		1.62
LZSS		1209277		0.91
LZSS + HA		820234		1.35
LZW		2188966		0.5
LZW + HA		1127646		0.98
Черно-белое изображение, размером не менее 800x600 пикселей				
	Исходный размер	После компрессии	После декомпрессии	Коэффициент сжатия
HA	1532562	1431696		1.07
RLE		24134		63.5
BWT + RLE		24134		63.5
BWT + MTF + HA		523159		2.93
BWT + MTF + RLE + HA		727468		2.1
LZSS		1521103		1.01

LZSS + HA		992529		1.54
LZW		3050038		0.5
LZW + HA		1767309		0.87
Изображение в оттенках серого, размером не менее 800x600 пикселей				
	Исходный размер	После компрессии	После декомпрессии	Коэффициент сжатия
HA	2001012	1851594	2001012	1.08
RLE		1156654		1.73
BWT + RLE		1419157		1.41
BWT + MTF + HA		1046101		1.91
BWT + MTF + RLE + HA		1212240		1.65
LZSS		2537206		0.79
LZSS + HA		1683745		1.19
LZW		3983236		0.5
LZW + HA		2337933		0.86
Цветное изображение, размером не менее 800x600 пикселей				
	Исходный размер	После компрессии	После декомпрессии	Коэффициент сжатия
HA	1806348	1747510	1806348	1.03
RLE		1824593		0.99
BWT + RLE		1505290		1.2
BWT + MTF + HA		1617967		1.12
BWT + MTF + RLE + HA		1676314		1.08
LZSS		3414002		0.53
LZSS + HA		2088378		0.86

LZW		3611638		0.5
LZW + HA		2127915		0.85

Ссылка на репозиторий Github с исходным кодом:

https://github.com/Name-pui/aads-comp-without-losses

Коды

```
def RLE_step3(b, Ms = 8, Mc = 8):

    new_b = b""
    n = 1
    i = 0
    symbol_size = Ms // 8
    control_size = Mc // 8
    while i < len(b) - symbol_size:
        bytes1 = b[i:i+symbol_size]
        bytes2 = b[i+symbol_size:i+2*symbol_size]
        if bytes1 == bytes2:
            n += 1
            i += symbol_size
        else:
            if n > 1:
                new_b += n.to_bytes(control_size, byteorder='big') + b[i:i+symbol_size]
                i += symbol_size
                n = 1
            else:
                buff = b""
                while i + symbol_size < len(b):
                    bytes1 = b[i:i+symbol_size]
                    bytes2 = b[i+symbol_size:i+2*symbol_size]
                    if bytes1 == bytes2:
                        break
                    buff += b[i:i+symbol_size]
                    i += symbol_size

                if i + symbol_size == len(b):
                    buff += b[i:i+symbol_size]
                    i += symbol_size

                symbol_count = len(buff) // symbol_size
                new_b += (symbol_count | 0x80).to_bytes(control_size, byteorder='big') + buff
                n = 1

    if i + symbol_size <= len(b):
        if n > 1:
```

```
            new_b += n.to_bytes(control_size, byteorder='big') + b[i:i+symbol_size]
        else:
            new_b += ((1 | 0x80).to_bytes(control_size, byteorder='big') + b[i:i+symbol_size])

    return new_b

def deRLE_step3(b, Ms = 8, Mc = 8):

    deB = b""
    i = 0
    symbol_size = Ms // 8
    control_size = Mc // 8
    while i < len(b):
        if b[i] & 0x80:
            count = b[i] & 0x7F
            deB += b[i+1:i+1+count*symbol_size]
            i += 1 + count * symbol_size
        else:
            deB += b[i+control_size:i+symbol_size+control_size]*b[i]
            i += 1 + symbol_size

    return deB

# Ms = 16
b = b"\xCF\xCE\xCF\xCE\xCF\xCE"
# Ms = 24
#b = b"\xCF\xCE\xCF\xCF\xCE\xCF"
print(b) # изначальная строка
new_b = RLE_step3(b)
print(new_b) # сжатая строка
deB = deRLE_step3(new_b)
print(deB) # разжатая строка
print('\n')

def read_rle_file(filename):
    with open(filename, 'rb') as f:
        meta = f.read(2)
        Ms, Mc = meta[0], meta[1]
        data = f.read()
    return Ms, Mc, data

def write_rle_file(filename, data, Ms, Mc):
    with open(filename, 'wb') as f:
        f.write(bytes([Ms, Mc]))
        f.write(data)

def compress_file_rle(input_path, output_path, Ms=16, Mc=16):
    with open(input_path, 'rb') as f:
        data = f.read()

    compressed = RLE_step3(data, Ms, Mc)
    write_rle_file(output_path, compressed, Ms, Mc)
```

```
def decompress_file_rle(input_path, output_path):
    Ms, Mc, compressed = read_rle_file(input_path)

    decompressed = deRLE_step3(compressed, Ms, Mc)

    with open(output_path, 'wb') as f:
        f.write(decompressed)

compress_file_rle("test2.txt", "test2_compression.txt")
decompress_file_rle("test2_compression.txt", "test2_compression_return.txt")

import math
def entropy(b, size):

    d = {}
    for i in range(0, len(b), size):
        block = b[i:i+size]
        d[block] = d.get(block, 0) + 1

    H = 0
    lenght = len(b) // size

    for i in d.values():
        if i > 0:
            p_i = i / lenght
            H -= p_i * math.log2(p_i)

    return H

print(entropy(b'abc', 1))
print(entropy(b'aaaa', 1))
print(entropy(b'aabb', 1))

def mtf_encode(b):
    alphabet = list(range(256))
    result = []

    for byte in b:
        index = alphabet.index(byte)
        result.append(index)
        alphabet.pop(index)
        alphabet.insert(0, byte)

    return bytes(result)

def mtf_decode(b):
    alphabet = list(range(256))
    result = []

    for index in b:
        byte = alphabet[index]
```

```
        result.append(byte)
        alphabet.pop(index)
        alphabet.insert(0, byte)

    return bytes(result)

original = b"aaaaaaaaaaaabbbbbbbcccccc"
encoded = mtf_encode(original)
decoded = mtf_decode(encoded)

print(f"Оригинал:  {original}")
print(f"MTF-код:   {encoded}")
print(f"Восстанов:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
print(f"Восстанов:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                {decoded}")

import heapq
def frequency_model(b):
    d = {}
    for i in b:
        d[i] = d.get(i, 0) + 1
    return d

def huffman_encode(data, model):
    # Сортируем
    queue = sorted([[char, freq] for char, freq in model.items()], key=lambda x: x[1])

    # Строим дерево
    while len(queue) > 1:
        one = queue.pop(0)
        two = queue.pop(0)
        parent = [None, one[1] + two[1], one, two]

        insert_position = 0
        for i, item in enumerate(queue):
            if parent[1] > item[1]:
                insert_position = i + 1
            else:
                break
        queue.insert(insert_position, parent)

    tree = queue[0] if queue else None

    # Кодируем
    codes = {}
    if tree:
        build_codes(tree, "", codes)

    # Кодируем данные
    encoded = ''.join(codes[char] for char in data)

    return codes, encoded, tree
```

```
def build_codes(node, prefix, codes):
    if len(node) == 2: # лист
        codes[node[0]] = prefix if prefix else "0"
        return

    if len(node) == 4: # узел
        build_codes(node[2], prefix + "0", codes)
        build_codes(node[3], prefix + "1", codes)



def huffman_decode(encoded, tree):
    if not tree or not encoded:
        return b""

    result = []
    node = tree
    for bit in encoded:
        if bit == '0':
            node = node[2]
        else:
            node = node[3]

        if len(node) == 2:
            result.append(node[0])
            node = tree

    return bytes(result)

def bits_to_bytes(bit_string):
    # Добавляем нули до кратности 8
    padding = (8 - len(bit_string) % 8) % 8
    bit_string += '0' * padding

    result = []
    for i in range(0, len(bit_string), 8):
        byte_chunk = bit_string[i:i+8]
        result.append(int(byte_chunk, 2))

    return bytes(result), padding



def bytes_to_bits(data, total_bits):
    bit_string = ''.join(f'{byte:08b}' for byte in data)
    return bit_string[:total_bits]

def read_huffman_file(filename):
    with open(filename, 'rb') as f:
        # Количество уникальных символов
        num_symbols = f.read(1)[0]

        # Таблица кодов
        codes = {}
```

```
        for _ in range(num_symbols):
            byte = f.read(1)[0]
            code_len = f.read(1)[0]
            code_byte = f.read(1)[0]
            code = f'{code_byte:08b}'[:code_len]
            codes[byte] = code

        # Служебная информация
        padding = f.read(1)[0]
        total_bits = int.from_bytes(f.read(4), 'big')

        # Сжатые данные
        compressed_data = f.read()
        encoded_bits = bytes_to_bits(compressed_data, total_bits)

    return codes, encoded_bits

def write_huffman_file(filename, codes, encoded_bits):
    compressed_bytes, padding = bits_to_bytes(encoded_bits)

    with open(filename, 'wb') as f:
        # Количество уникальных символов
        f.write(bytes([len(codes)]))

        # Таблица кодов
        for byte, code in codes.items():
            f.write(bytes([byte]))          # символ
            f.write(bytes([len(code)]))     # длина кода
            code_padded = code.ljust(8, '0')
            f.write(bytes([int(code_padded, 2)]))  # код

        # Служебная информация
        f.write(bytes([padding]))
        f.write(len(encoded_bits).to_bytes(4, 'big'))

        # Сжатые данные
        f.write(compressed_bytes)

    print(f"Записано в {filename}: {len(compressed_bytes) + 7 + len(codes)*3} байт")

def compress_file(input_path, output_path):
    with open(input_path, 'rb') as f:
        data = f.read()

    model = frequency_model(data)
    codes, encoded_bits, tree = huffman_encode(data, model)
    write_huffman_file(output_path, codes, encoded_bits)

    print(f"Оригинал: {len(data)} байт")


def decompress_file(input_path, output_path):
```

```
    codes, encoded_bits = read_huffman_file(input_path)

    # Восстанавливаем дерево из кодов
    reverse_codes = {v: k for k, v in codes.items()}

    result = []
    buffer = ""
    for bit in encoded_bits:
        buffer += bit
        if buffer in reverse_codes:
            result.append(reverse_codes[buffer])
            buffer = ""

    with open(output_path, 'wb') as f:
        f.write(bytes(result))

    print(f"Распаковано: {len(result)} байт")

import struct
def bwt_encode_block(b):
    n = len(b)
    matrix = [b[i:] + b[:i] for i in range(n)]
    matrix.sort()
    orig_index = matrix.index(b)
    last_column = bytes([i[-1] for i in matrix])

    return last_column, orig_index

def bwt_decode_block(last_column, index):
    n = len(last_column)

    count = [0]*256
    for byte in last_column:
        count[byte] += 1

    total = 0
    for i in range(256):
        c = count[i]
        count[i] = total
        total += c

    next_pos = [0]*n
    current_count = [0]*256
    for i in range(n):
        byte = last_column[i]
        pos = count[byte] + current_count[byte]
        next_pos[i] = pos
        current_count[byte] += 1

    result = bytearray()
    row = index
    for i in range(n):
```

```
        byte = last_column[row]
        result.append(byte)
        row = next_pos[row]

    return bytes(result[::-1])

def bwt_encode(data, block_size=None):
    if block_size is None:
        return bwt_encode_block(data)

    num_blocks = (len(data) + block_size - 1) // block_size
    output = struct.pack('<I', num_blocks)

    for i in range(0, len(data), block_size):
        chunk = data[i:i+block_size]
        col, idx = bwt_encode_block(chunk)
        output += struct.pack('<I', idx)
        output += struct.pack('<I', len(col))
        output += col

    return output

def bwt_decode(data, index=None, blocked=False):
    if not blocked:
        return bwt_decode_block(data, index)

    offset = 0
    num_blocks = struct.unpack('<I', data[offset:offset+4])[0]
    offset += 4

    result = bytearray()
    for _ in range(num_blocks):
        idx = struct.unpack('<I', data[offset:offset+4])[0]
        length = struct.unpack('<I', data[offset+4:offset+8])[0]
        offset += 8

        block_data = data[offset:offset+length]
        offset += length

        result.extend(bwt_decode_block(block_data, idx))

    return                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                bytes(result)

def suffix_array_to_bwt(suffix_array, text):
    n = len(text)
    result = bytearray()
    for pos in suffix_array:
        prev_pos = (pos - 1) % n
        result.append(text[prev_pos])
```

```
    return bytes(result)

def suffix_array_to_bwt(suffix_array, text):
    n = len(text)
    result = bytearray()
    for pos in suffix_array:
        prev_pos = (pos - 1) % n
        val = text[prev_pos]
        if isinstance(val, str):
            val = ord(val)
        result.append(val)
    return bytes(result)

def build_suffix_array(text):
    n = len(text)
    if n == 0: return []

    # Приводим к числам для удобства
    nums = [ord(c) for c in text] if isinstance(text, str) else list(text)

    sa = list(range(n))
    # Начальная сортировка по первому символу
    sa.sort(key=lambda i: nums[i])

    rank = [0] * n
    rank[sa[0]] = 0
    for i in range(1, n):
        rank[sa[i]] = rank[sa[i-1]] + (1 if nums[sa[i]] != nums[sa[i-1]] else 0)

    k = 1
    while k < n:
        # Сортировка по парам рангов (текущий, следующий через k)
        sa.sort(key=lambda i: (rank[i], rank[(i + k) % n]))

        new_rank = [0] * n
        new_rank[sa[0]] = 0
        for i in range(1, n):
            prev, curr = sa[i-1], sa[i]
            pair_prev = (rank[prev], rank[(prev + k) % n])
            pair_curr = (rank[curr], rank[(curr + k) % n])
            new_rank[curr] = new_rank[prev] + (1 if pair_prev != pair_curr else 0)

        rank = new_rank
        if rank[sa[n-1]] == n - 1: break # Все ранги уникальны
        k *= 2

    return sa

def bwt_transform(text):
    # Основная функция преобразования.
    if isinstance(text, bytes):
        text = text.decode('utf-8')
```

```
    sa = build_suffix_array(text)
    return suffix_array_to_bwt(sa, text)

s = "banana"
res = bwt_transform(s)
print(f"Text: {s}")
print(f"BWT:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                {res.decode('utf-8')}")

def lz77_encode(data: bytes, window_size: int = 4096, lookahead_size: int = 18):
    if not data:
        return b"

    encoded = bytearray()
    n = len(data)
    pos = 0

    while pos < n:
        # Определяем границы буфера поиска
        search_start = max(0, pos - window_size)
        search_end = pos
        search_buffer = data[search_start:search_end]

        # Ищем максимальное совпадение
        best_offset = 0
        best_length = 0

        # Максимальная длина совпадения ограничена буфером просмотра
        max_lookahead = min(lookahead_size, n - pos)

        # Перебираем возможные длины совпадения
        for length in range(1, max_lookahead + 1):
            # Ищем подстроку data[pos:pos+length] в буфере поиска
            pattern = data[pos:pos + length]

            # Поиск справа налево для нахождения ближайшего совпадения
            offset = search_buffer.rfind(pattern)

            if offset != -1:
                # Найдено совпадение
                # offset отсчитывается от конца search_buffer
                actual_offset = len(search_buffer) - offset
                if actual_offset > best_length or length > best_length:
                    best_offset = actual_offset
                    best_length = length
            else:
                break

        # Определяем следующий символ
        next_pos = pos + best_length
        next_byte = data[next_pos] if next_pos < n else 0

        # Кодируем тройку (смещение, длина, символ)
```

```
        # Используем 2 байта для смещения и 2 байта для длины
        encoded.extend(best_offset.to_bytes(2, 'big'))
        encoded.extend(best_length.to_bytes(2, 'big'))
        encoded.append(next_byte)

        # Сдвигаем окно
        pos += best_length + 1

    return bytes(encoded)


def lz77_decode(encoded: bytes):
    decoded = bytearray()
    i = 0
    n = len(encoded)

    while i < n:
        # Читаем тройку: смещение (2 байта), длина (2 байта), символ (1 байт)
        if i + 5 > n:
            break

        offset = int.from_bytes(encoded[i:i+2], 'big')
        length = int.from_bytes(encoded[i+2:i+4], 'big')
        next_byte = encoded[i+4]
        i += 5

        # Копируем совпадение из уже декодированных данных
        if length > 0:
            start_pos = len(decoded) - offset
            for j in range(length):
                decoded.append(decoded[start_pos + j])

        # Добавляем следующий символ
        decoded.append(next_byte)

    return bytes(decoded)

print("OK" if lz77_decode(lz77_encode(b"banana")) == b"banana" else "FAIL")

def lzss_encode(data: bytes, window_size: int = 4096, lookahead_size: int = 18, min_match: int = 3):
    if not data:
        return b"

    encoded = bytearray()
    n = len(data)
    pos = 0

    while pos < n:
        search_start = max(0, pos - window_size)
        search_buffer = data[search_start:pos]
        max_lookahead = min(lookahead_size, n - pos)
```

```
        best_offset = 0
        best_length = 0

        for length in range(min_match, max_lookahead + 1):
            pattern = data[pos:pos + length]
            offset = search_buffer.rfind(pattern)

            if offset != -1:
                actual_offset = len(search_buffer) - offset
                if actual_offset <= window_size:
                    best_offset = actual_offset
                    best_length = length
            else:
                break

        if best_length >= min_match:
            # Флаг 0x80 + смещение (2 байта) + длина (1 байт)
            encoded.append(0x80 | ((best_offset >> 8) & 0x7F))
            encoded.append(best_offset & 0xFF)
            encoded.append(best_length)
            pos += best_length
        else:
            # Флаг 0x00 + литерал
            encoded.append(0x00)
            encoded.append(data[pos])
            pos += 1

    return bytes(encoded)


def lzss_decode(encoded: bytes):
    decoded = bytearray()
    i = 0
    n = len(encoded)

    while i < n:
        flag = encoded[i]
        i += 1

        if flag == 0:
            # Литерал
            if i >= n:
                break
            decoded.append(encoded[i])
            i += 1
        else:
            # Ссылка
            if i + 2 > n:
                break
            offset = ((flag & 0x7F) << 8) | encoded[i]
            length = encoded[i + 1]
            i += 2
```

```
        start_pos = len(decoded) - offset
        for j in range(length):
            decoded.append(decoded[start_pos + j])

    return bytes(decoded)


data = b"banana"
encoded = lzss_encode(data)
decoded = lzss_decode(encoded)
print(f"LZSS: {decoded == data}")

def lz78_encode(data: bytes, max_dict_size: int = 4096):
    if not data:
        return b"

    encoded = bytearray()
    n = len(data)
    pos = 0

    # Словарь: фраза -> индекс
    dictionary = {b"": 0}
    next_index = 1

    current_phrase = b""

    while pos < n:
        # Добавляем следующий байт
        current_byte = bytes([data[pos]])
        new_phrase = current_phrase + current_byte

        if new_phrase in dictionary and next_index < max_dict_size:
            # Фраза есть в словаре, продолжаем
            current_phrase = new_phrase
            pos += 1
        else:
            # Выдаем пару (индекс префикса, текущий байт)
            prefix_index = dictionary[current_phrase] if current_phrase else 0

            # Кодируем индекс (2 байта) и символ (1 байт)
            encoded.extend(prefix_index.to_bytes(2, 'big'))
            encoded.append(data[pos])

            # Добавляем новую фразу в словарь (если не пустая и не превышен лимит)
            if new_phrase and next_index < max_dict_size:
                dictionary[new_phrase] = next_index
                next_index += 1

            # Сбрасываем текущую фразу
            current_phrase = b""
            pos += 1
```

```
    # Обработка остатка (если текущая фраза не пуста)
    if current_phrase:
        prefix_index = dictionary[current_phrase]
        encoded.extend(prefix_index.to_bytes(2, 'big'))
        encoded.append(0)

    return bytes(encoded)


def lz78_decode(encoded: bytes, max_dict_size: int = 4096):
    if not encoded:
        return b''

    decoded = bytearray()
    n = len(encoded)
    i = 0

    # Словарь: индекс -> фраза
    dictionary = {0: b''}
    next_index = 1

    while i + 2 < n:
        # Читаем индекс (2 байта) и символ (1 байт)
        index = int.from_bytes(encoded[i:i+2], 'big')
        next_byte = encoded[i+2]
        i += 3

        # Получаем фразу по индексу
        prefix = dictionary.get(index, b'')

        # Формируем новую фразу
        if next_byte == 0:
            new_phrase = prefix
        else:
            new_phrase = prefix + bytes([next_byte])

        # Добавляем в декодированный вывод
        decoded.extend(new_phrase)

        # Добавляем в словарь, если не превышен лимит
        if new_phrase and next_index < max_dict_size:
            dictionary[next_index] = new_phrase
            next_index += 1

    return bytes(decoded)

def lzw_encode(data: bytes, max_dict_size: int = 4096):
    if not data:
        return b''

    encoded = bytearray()
```

```
    n = len(data)
    pos = 0

    # Инициализируем словарь всеми возможными байтами
    dictionary = {bytes([i]): i for i in range(256)}
    next_index = 256

    current_phrase = b""

    while pos < n:
        # Добавляем следующий байт
        current_byte = bytes([data[pos]])
        new_phrase = current_phrase + current_byte

        if new_phrase in dictionary and next_index < max_dict_size:
            # Фраза есть в словаре, продолжаем
            current_phrase = new_phrase
            pos += 1
        else:
            # Выдаем индекс текущей фразы
            prefix_index = dictionary[current_phrase]
            encoded.extend(prefix_index.to_bytes(2, 'big'))

            # Добавляем новую фразу в словарь (если не превышен лимит)
            if new_phrase and next_index < max_dict_size:
                dictionary[new_phrase] = next_index
                next_index += 1

            # Сбрасываем текущую фразу
            current_phrase = current_byte
            pos += 1

    # Обработка остатка
    if current_phrase:
        prefix_index = dictionary[current_phrase]
        encoded.extend(prefix_index.to_bytes(2, 'big'))

    return bytes(encoded)


def lzw_decode(encoded: bytes, max_dict_size: int = 4096):
    if not encoded:
        return b"

    decoded = bytearray()
    n = len(encoded)
    i = 0

    # Инициализируем словарь всеми возможными байтами
    dictionary = {i: bytes([i]) for i in range(256)}
    next_index = 256
```

```
    # Читаем первый индекс
    if i + 2 > n:
        return b''

    prev_index = int.from_bytes(encoded[i:i+2], 'big')
    i += 2

    # Первая фраза
    prev_phrase = dictionary.get(prev_index, b'')
    decoded.extend(prev_phrase)

    while i + 2 <= n:
        # Читаем следующий индекс
        curr_index = int.from_bytes(encoded[i:i+2], 'big')
        i += 2

        # Получаем фразу
        if curr_index in dictionary:
            curr_phrase = dictionary[curr_index]
        elif curr_index == next_index and next_index < max_dict_size:
            # Специальный случай: фраза, которую только будем добавлять
            curr_phrase = prev_phrase + bytes([prev_phrase[0]])
        else:
            # Некорректный индекс
            break

        # Добавляем в декодированный вывод
        decoded.extend(curr_phrase)

        # Добавляем новую фразу в словарь
        if next_index < max_dict_size and prev_phrase:
            new_phrase = prev_phrase + bytes([curr_phrase[0]])
            dictionary[next_index] = new_phrase
            next_index += 1

        prev_phrase = curr_phrase

    return bytes(decoded)

import os
def get_code_lengths(node, length=0, lengths=None):
    # Обход дерева для получения длин кодов для каждого символа
    if lengths is None:
        lengths = {}

    if len(node) == 2:  # Лист (символ, частота)
        # Обработка случая одного символа
        lengths[node[0]] = max(1, length)
        return lengths

    if len(node) == 4:  # Узел (None, вес, лево, право)
        get_code_lengths(node[2], length + 1, lengths)
```

```
        get_code_lengths(node[3], length + 1, lengths)

    return lengths

def generate_canonical_codes(lengths):
    # Сортируем символы: сначала по длине кода, затем по значению символа
    sorted_symbols = sorted(lengths.keys(), key=lambda s: (lengths[s], s))

    codes = {}
    code = 0
    prev_len = 0

    for symbol in sorted_symbols:
        curr_len = lengths[symbol]
        # Сдвигаем код на разницу длин
        code <<= (curr_len - prev_len)

        # Формируем битовую строку
        codes[symbol] = format(code, f'0{curr_len}b')

        code += 1
        prev_len = curr_len

    return codes

def frequency_model(b):
    d = {}
    for i in b:
        d[i] = d.get(i, 0) + 1
    return d

def huffman_encode(data, model):
    # Сортируем очередь по частоте
    queue = sorted([[char, freq] for char, freq in model.items()], key=lambda x: x[1])

    # Строим дерево
    while len(queue) > 1:
        one = queue.pop(0)
        two = queue.pop(0)
        # Узел: [None, вес, левый_потомок, правый_потомок]
        parent = [None, one[1] + two[1], one, two]

        # Вставляем обратно, сохраняя сортировку по весу
        insert_position = 0
        for i, item in enumerate(queue):
            if parent[1] > item[1]:
                insert_position = i + 1
            else:
                break
        queue.insert(insert_position, parent)

    tree = queue[0] if queue else None
```

```
    # 1. Получаем длины кодов из дерева
    lengths = get_code_lengths(tree) if tree else {}

    # 2. Генерируем канонические коды по длинам
    codes = generate_canonical_codes(lengths)

    # 3. Кодируем данные
    encoded = ''.join(codes[char] for char in data)

    return codes, encoded, lengths

def huffman_decode(encoded, lengths):
    if not lengths or not encoded:
        return b''

    # Восстанавливаем канонические коды для декодирования
    codes = generate_canonical_codes(lengths)
    reverse_codes = {v: k for k, v in codes.items()}

    result = []
    buffer = ''

    # Оптимизация: максимальная длина кода для ограничения поиска
    max_len = max(lengths.values()) if lengths else 0

    for bit in encoded:
        buffer += bit
        # Проверяем, есть ли такой код в таблице
        if len(buffer) <= max_len and buffer in reverse_codes:
            result.append(reverse_codes[buffer])
            buffer = ''

    return bytes(result)

def bits_to_bytes(bit_string):
    if not bit_string:
        return b'', 0

    padding = (8 - len(bit_string) % 8) % 8
    bit_string += '0' * padding

    result = []
    for i in range(0, len(bit_string), 8):
        byte_chunk = bit_string[i:i+8]
        result.append(int(byte_chunk, 2))

    return bytes(result), padding

def bytes_to_bits(data, total_bits):
    bit_string = ''.join(f'{byte:08b}' for byte in data)
    return bit_string[:total_bits]
```

```
def write_huffman_file(filename, codes, encoded_bits, lengths):
    compressed_bytes, padding = bits_to_bytes(encoded_bits)
    unique_symbols = len(codes)

    with open(filename, 'wb') as f:
        # 1. Количество уникальных символов
        f.write(bytes([unique_symbols]))

        # 2. Таблица: Символ + Длина кода
        for byte_val in sorted(codes.keys()):
            f.write(bytes([byte_val]))
            f.write(bytes([lengths[byte_val]]))

        # 3. Служебная информация
        f.write(bytes([padding]))
        f.write(len(encoded_bits).to_bytes(4, 'big'))

        # 4. Сжатые данные
        f.write(compressed_bytes)

    header_size = 1 + (unique_symbols * 2) + 1 + 4
    print(f"Записано в {filename}: {len(compressed_bytes) + header_size} байт (Заголовок: {header_size})")

def read_huffman_file(filename):
    with open(filename, 'rb') as f:
        # 1. Количество уникальных символов
        num_symbols = f.read(1)[0]

        # 2. Читаем длины кодов
        lengths = {}
        for _ in range(num_symbols):
            byte_val = f.read(1)[0]
            code_len = f.read(1)[0]
            lengths[byte_val] = code_len

        # 3. Служебная информация
        padding = f.read(1)[0]
        total_bits = int.from_bytes(f.read(4), 'big')

        # 4. Сжатые данные
        compressed_data = f.read()
        encoded_bits = bytes_to_bits(compressed_data, total_bits)

    return lengths, encoded_bits

def compress_file(input_path, output_path):
    with open(input_path, 'rb') as f:
        data = f.read()

    if not data:
        with open(output_path, 'wb') as f:
```

```
            f.write(bytes([0])) # 0 символов
        return

    model = frequency_model(data)
    codes, encoded_bits, lengths = huffman_encode(data, model)
    write_huffman_file(output_path, codes, encoded_bits, lengths)

    print(f"Оригинал: {len(data)} байт")

def decompress_file(input_path, output_path):
    lengths, encoded_bits = read_huffman_file(input_path)

    if not lengths:
        with open(output_path, 'wb') as f:
            pass
        return

    result = huffman_decode(encoded_bits, lengths)

    with open(output_path, 'wb') as f:
        f.write(result)

    print(f"Распаковано: {len(result)} байт")
```